

- RAG Learning

2.1 Elements of Computational Thinking

Computer Science — OCR — Component 02 (Algorithms and Programming)

Computational thinking is **a set of mental techniques used by computer scientists to formulate a problem in a way that a computer can solve it**. OCR's 2.1 specification breaks it into **five interrelated elements**: thinking *abstractly*, *ahead*, *procedurally*, *logically* and *concurrently*. These are not steps in a recipe — they are complementary mindsets that a developer moves between fluidly while designing, coding and testing a system.

Computational thinking is the process of analysing and solving problems using techniques drawn from computer science, such that the solution could (in principle) be carried out by a computer or a human. It emphasises problem *formulation* as much as problem *coding*.

2.1.1 Thinking Abstractly

Abstraction is the process of **removing or hiding unnecessary detail** so that the solver can focus on the features of a problem that matter. The result — the **abstract model** — is a simplified representation of reality.

Abstraction is widely regarded as the single most important idea in computing. Every useful layer of a computer system — high-level languages, operating systems, network protocols, databases — exists because someone abstracted away the messy detail of the layer below. Without abstraction, a programmer writing a web page would have to think about voltage levels on transistors; with it, they think in terms of HTML tags.

Types of Abstraction

Type	What it hides	Example
Representational abstraction	Physical or geographical detail not needed to use the model	The London Underground map: it preserves <i>connectivity</i> and <i>order of stations</i> but distorts distances and directions. It is topological , not geographical.
Abstraction by generalisation	The specifics that differ between similar problems	Noticing that "find the shortest delivery route", "find the fastest journey on the tube" and "find the cheapest airline connection" are all instances of a weighted-graph shortest-path problem.
Procedural abstraction	The implementation details behind a named procedure	A programmer calls <code>sort(list)</code> without needing to know whether it uses merge sort, quicksort or a hybrid; the routine is a "black box" with a defined interface.
Data abstraction	The internal storage mechanism of a data structure	A stack is characterised by <code>push</code> / <code>pop</code> / <code>peek</code> operations and LIFO behaviour; whether it is built on an array or a linked list is irrelevant to its user.
Functional abstraction	How a function achieves its result	Using <code>sqrt(x)</code> without caring about Newton's method, CORDIC or a lookup table.

Representational abstraction: the Tube map

Harry Beck's 1933 London Underground map is the canonical example. The real tunnels curve and branch unevenly, station spacing varies enormously, and the network covers thousands of square kilometres. A commuter does not care about any of that — they need to know "which line do I get on, do I change anywhere, which direction?". Beck's diagram keeps only that relational information. It is *wrong* geographically (stations appear equidistant; the Thames is a schematic curve) but *right* functionally: it answers the user's question with minimum cognitive load.

Exam Tip: A good exam answer distinguishes **what is removed** (exact distances, curvature, geography) from **what is preserved** (station order, line colour, interchanges). Always state both.

Procedural and data abstraction in code

When a programmer writes:

```
result = shortest_path(graph, start, end)
```

they are using both forms of abstraction at once: **procedural** (the algorithm that computes the path is hidden behind the function name) and **data** (the graph is treated as a conceptual structure of nodes and edges, regardless of whether it is stored as an adjacency list, adjacency matrix or on-disk database).

Worked Example — Abstracting a Library

Asked to "design a library system", a student begins with **representational abstraction**: what are the essential entities and relationships, ignoring buildings, paint colour, opening hours and physical shelving?

Entity	Essential attributes	Relationship
Book	title, author, ISBN, copy count	borrowed by → Member
Member	member ID, name, contact, join date	borrowes → Book; reserves → Book
Loan	book copy, member, issue date, due date	links Book ↔ Member
Reservation	book, member, date placed	links Book ↔ Member

Notice what is *omitted*: shelf positions, the librarian's coffee preferences, the building's fire exits, whether the database runs on MySQL or PostgreSQL. These are irrelevant to modelling how borrowing works. The result is a conceptual *data model* that can later be refined into a relational schema, an object hierarchy, or a REST API — but the abstract model is independent of all of those.

Exam Tip: When an OCR question asks you to "describe how abstraction has been used", do not just list features. Explicitly state a removed detail (e.g. "the geographical distance between stations is removed") and a retained detail (e.g. "the order of stations on each line is kept"), then say *why* the removal is appropriate for the intended user.

Why Abstraction is Powerful

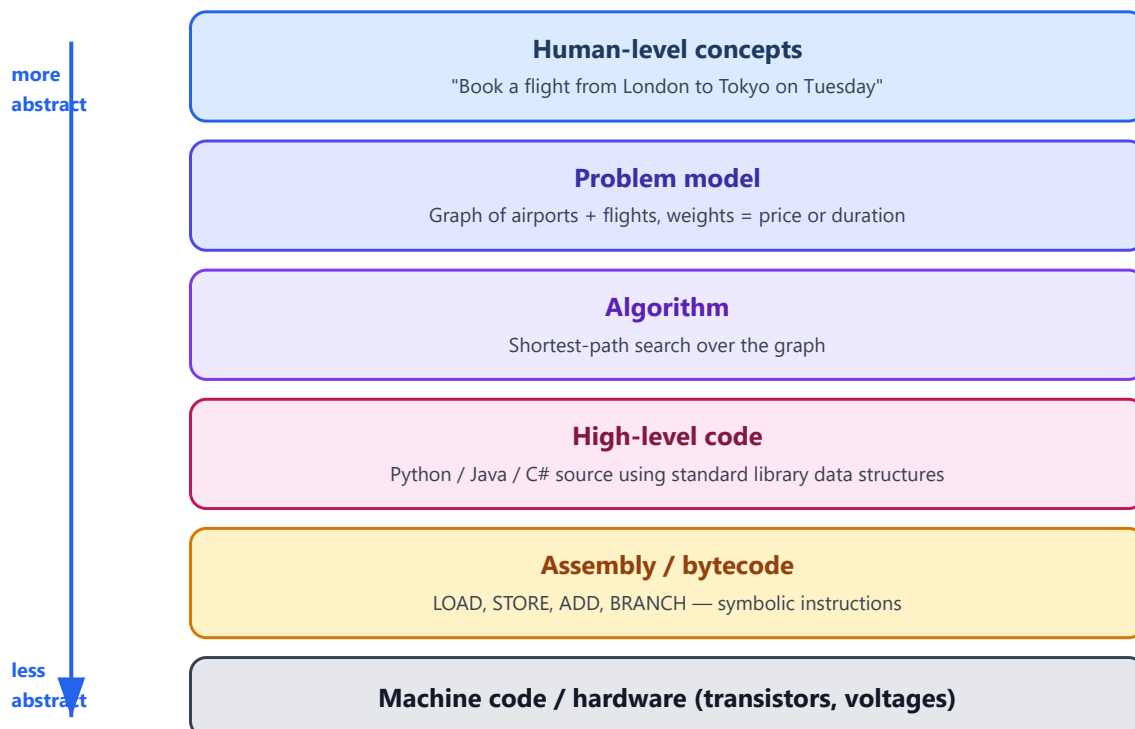
Three benefits make abstraction the most reused tool in computing:

- **Cognitive load reduction:** the solver only holds the relevant detail in working memory. A programmer using `sort()` does not have to remember the quicksort partition invariant.
- **Substitutability:** so long as the abstraction's promises (interface, behaviour) are kept, the implementation can change without breaking the caller. A database can move from SQLite to PostgreSQL without every query needing a rewrite.
- **Composition:** small abstractions combine to form larger ones. An abstract "shape" is used to build an abstract "drawing", which is used to build an abstract "document editor".

Levels of Abstraction

Software is built as a stack of abstractions, each layer presenting a simpler interface to the layer above it. This is why it is practical for one developer to write a chess game without also designing a CPU.

Levels of Abstraction — from Human Concepts to Hardware



Each arrow downwards represents a *removal of abstraction*: more detail is added, the representation becomes more concrete, and the programmer's freedom to ignore implementation decreases. Each

arrow upwards is an *application of abstraction*: detail is suppressed to allow higher-level reasoning.

Exam Tip: Candidates frequently confuse **abstraction** (remove irrelevant detail to simplify a model) with **decomposition** (break a problem into smaller sub-problems). They are complementary but different: you can abstract without decomposing (e.g. the Tube map) and decompose without abstracting (chopping a task into sub-tasks while keeping every real-world detail).

2.1.2 Thinking Ahead

Thinking ahead is anticipating what a program (and its users) will need before writing code: the **inputs** required, the **outputs** expected, the **preconditions** that must hold, the **edge cases** that may arise, and the **reusable results** that can be cached or precomputed.

Determining Inputs and Outputs

Before designing any procedure, the developer lists explicitly:

- **Inputs:** every piece of data the procedure needs — source, type, range, format.
- **Outputs:** every piece of data the procedure produces — destination, type, format, accuracy.
- **Side effects** (often overlooked): writes to a file, emails sent, global state mutated.

Failing to do this produces code that "almost works" — it handles the developer's test data but breaks on real-world input.

Preconditions

A **precondition** is a statement that must be true *before* a procedure is invoked for the procedure to work correctly. If the caller violates the precondition, the procedure's behaviour is undefined.

Example: a procedure `withdraw(account, amount)` might state preconditions:

- the account exists and is not frozen;
- `amount` is positive and $\leq$ the current balance;
- the caller holds the account lock (in a concurrent system).

Making these explicit up-front prevents defensive code being scattered across every call-site and allows each procedure to be reasoned about in isolation.

Why Preconditions Matter in Big Systems

Explicit preconditions become increasingly important as a system grows. In a single-file script, the author remembers every invariant. In a 500,000-line codebase with 30 contributors, the only way anyone can safely call a procedure is if its preconditions are *stated* — ideally in comments or type annotations, and ideally checked by an automated test. Preconditions are therefore not merely a design nicety; they are a communication mechanism between today's code author and tomorrow's maintainer.

Caching and Reusability

Caching is storing the result of a computation (or an I/O request) so that future identical requests can be served from the cached copy instead of being recomputed or re-fetched.

Memoisation is the specific case of caching the results of pure functions keyed by their arguments.

Caching is a forward-thinking optimisation: the developer anticipates that the same expensive work will be repeated and chooses to pay for memory to save time. Common forms include:

- **Memoisation**: wrapping a pure function so that repeated calls with the same arguments return the stored result.
- **Precomputed lookup tables**: values computed once at build-time and embedded in the program (e.g. sine tables for graphics, CRC tables for error detection).
- **HTTP caches**: browsers storing downloaded images so revisits do not re-download.
- **CPU caches (L1/L2/L3)**: hardware-level caching already studied in 1.1.

Edge Cases and Exceptions

An experienced developer thinks ahead to the unusual inputs that will inevitably arrive:

- **Boundary values:** 0, 1, maximum size, empty collections.
- **Invalid inputs:** malformed strings, out-of-range numbers, null/None values.
- **Concurrent access:** two users clicking "book" on the last seat at the same instant.
- **Resource failure:** network loss, disk full, database down.
- **Locale:** non-ASCII text, right-to-left scripts, 24-hour vs 12-hour clocks.

Forward-thinking design decisions

Thinking ahead also spans the *life of the system*: will the user base grow tenfold? will the data volumes outgrow a single machine? will a new regulation require encryption at rest? Such concerns influence the initial architecture (choice of database, statelessness of services, separation of logic from UI) even when they are not yet active requirements.

Worked Example — A Weather App Team Thinking Ahead

A small team is designing a mobile weather app. Before writing code they enumerate what they will need to anticipate:

Concern	Decision taken ahead of implementation
API rate limit: the provider allows 500 calls/hour/user	Cache the last response for each location for 15 minutes on the device — most re-opens will be served from cache, well within the limit.
Offline / no-signal use	Persist the last successful response to local storage; on launch, show the cached data immediately with a timestamp, then refresh in the background.
Time zone mismatch: user in Tokyo checking New York weather	Store all forecast timestamps as UTC internally; convert to the target location's local time in the UI. Never mix zones in the same comparison.
Locale	Drive Celsius/Fahrenheit from device settings, not hard-coded. Use Unicode throughout.

Growth	Design the API client so the provider can be swapped without touching the UI layer (procedural abstraction supports this).
Failure	Decide up-front what to display if the API is unreachable and the cache is empty — a clear error state rather than a blank screen.

Each of these is an act of thinking ahead: identifying a future condition and making a design decision *now* that will handle it gracefully *later*. None of these changes the core algorithm (fetch → parse → display); they surround it with anticipatory structure.

Exam Tip: When asked to "describe how a developer might think ahead when designing system X", structure your answer around the OCR list: inputs & outputs; preconditions; caching / reusability; edge cases; scalability. Give one concrete example per heading and you will score across the mark scheme.

2.1.3 Thinking Procedurally

Thinking procedurally is identifying the **individual steps** that make up a solution, their **order**, and the **sub-procedures** that are called in the process. It focuses on *how* a task is carried out rather than *what* conditions apply.

Procedural thinking answers the question "what needs to happen, in what sequence?". It is the mindset most directly corresponding to writing the body of a function: first do A, then B, then call C (which itself does D then E). The big tools of procedural thinking are **decomposition** and the dual strategies of **top-down** and **bottom-up** design.

Decomposition

Decomposition is breaking a complex problem down into smaller, more manageable sub-problems that can each be solved independently and whose solutions combine to solve the original. It is a divide-and-conquer mindset applied at design time.

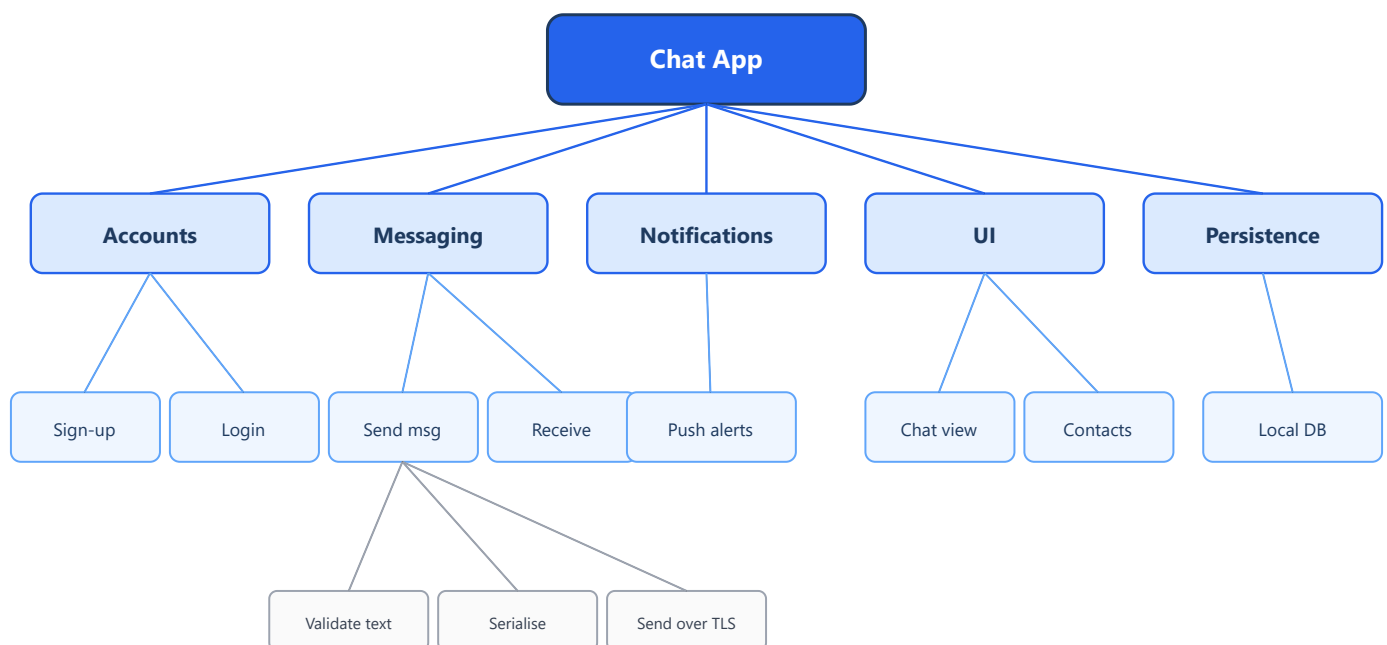
Decomposition is valuable because:

- **Each sub-problem is simpler** and often already solved (re-use existing libraries / patterns).
- **Work can be shared** across a team — different developers own different modules.
- **Each sub-problem can be tested in isolation**, making bugs easier to locate.
- **Modules can be replaced** without rewriting the whole system.

Worked Example — Decomposing "Build a Chat App"

"Build a chat app" is far too large to start coding. A developer decomposes it:

Decomposition Tree — "Build a Chat App"



Each leaf is a unit small enough to be implemented and tested independently.

The act of drawing this tree is decomposition. Each level answers the question "what smaller things would we have to build to build this?". At some leaf depth a box becomes "small enough" to be assigned to a developer and implemented directly.

Top-down (Stepwise Refinement) vs Bottom-up Design

Top-down design (stepwise refinement): start with a high-level statement of the whole problem, then repeatedly break each step into smaller steps until each is trivial to implement.

Decomposition from the root downwards.

Bottom-up design: start by building small, reliable, reusable components (often already existing as libraries), then compose them upwards into ever larger modules until the complete system emerges.

Aspect	Top-down	Bottom-up
Starting point	The full problem as a single statement	Small, known pieces (libraries, utility functions)
Strength	Keeps overall architecture clear; every module exists for a reason	Promotes reuse; each component can be thoroughly tested before it is needed
Risk	May design modules whose leaves turn out to be hard or impossible	The composition at the top may not match what was actually needed
Typical use	Structured design in greenfield projects; OCR algorithm-design questions	Building on existing frameworks; library-driven development

Real projects blend both: a top-down sketch gives direction, while bottom-up library reuse gives leverage. A chat app designer may use top-down decomposition overall, but at the leaves reaches for bottom-up reuse of, say, an existing TLS library.

Procedural steps and sub-procedures

Thinking procedurally is not just decomposing — it also means deciding *the order* in which steps run and *which steps become reusable sub-procedures*. If two branches of the tree both need "validate user input", that becomes one shared procedure called from each, not duplicated code. Identifying shared sub-procedures is where procedural thinking blurs productively into **procedural abstraction** (see 2.1.1).

Exam Tip: Procedural thinking and decomposition are tightly linked. When OCR asks for a *decomposition diagram*, draw a tree with exactly the shapes above: a root box, branches to

sub-problems, sub-sub-problems where needed. Label each box with a verb phrase ("Validate user input", not just "Input").

2.1.4 Thinking Logically

Thinking logically is identifying the **decision points** in a problem and how they cause the program to follow different paths. It focuses on Boolean conditions, branching and the logical structure of choice.

Where thinking procedurally answers "what steps, in what order?", thinking logically answers "*under what conditions* do we take each step?". Every `if`, `while` and `case` in a program is the result of a logical-thinking decision.

Decision Points and Boolean Conditions

A **decision point** is any place in a program where execution can take more than one path. It evaluates a Boolean expression and selects a branch accordingly. Typical sources of decision points:

- User actions (keyboard input, mouse clicks)
- Data validation (is the input in range? is the account active?)
- Authorisation (does this user have permission?)
- State-machine transitions (is the game paused / running / finished?)
- Error handling (did the network request succeed?)

Worked Example — Decision Tree for a Game Turn

Consider a grid-based strategy game where it is the player's turn to move a unit. The decision tree is:

```
Is it this player's turn?
  No → ignore input, wait
  Yes → Has a unit been selected?
        No → is the click on a friendly unit?
              Yes → select it
              No → do nothing
```

```
Yes → is the clicked square a valid move target?  
    No → play error sound, deselect  
    Yes → does the square contain an enemy?  
        Yes → resolve attack, update HP  
        No → move unit to square  
            update turn state, end turn
```

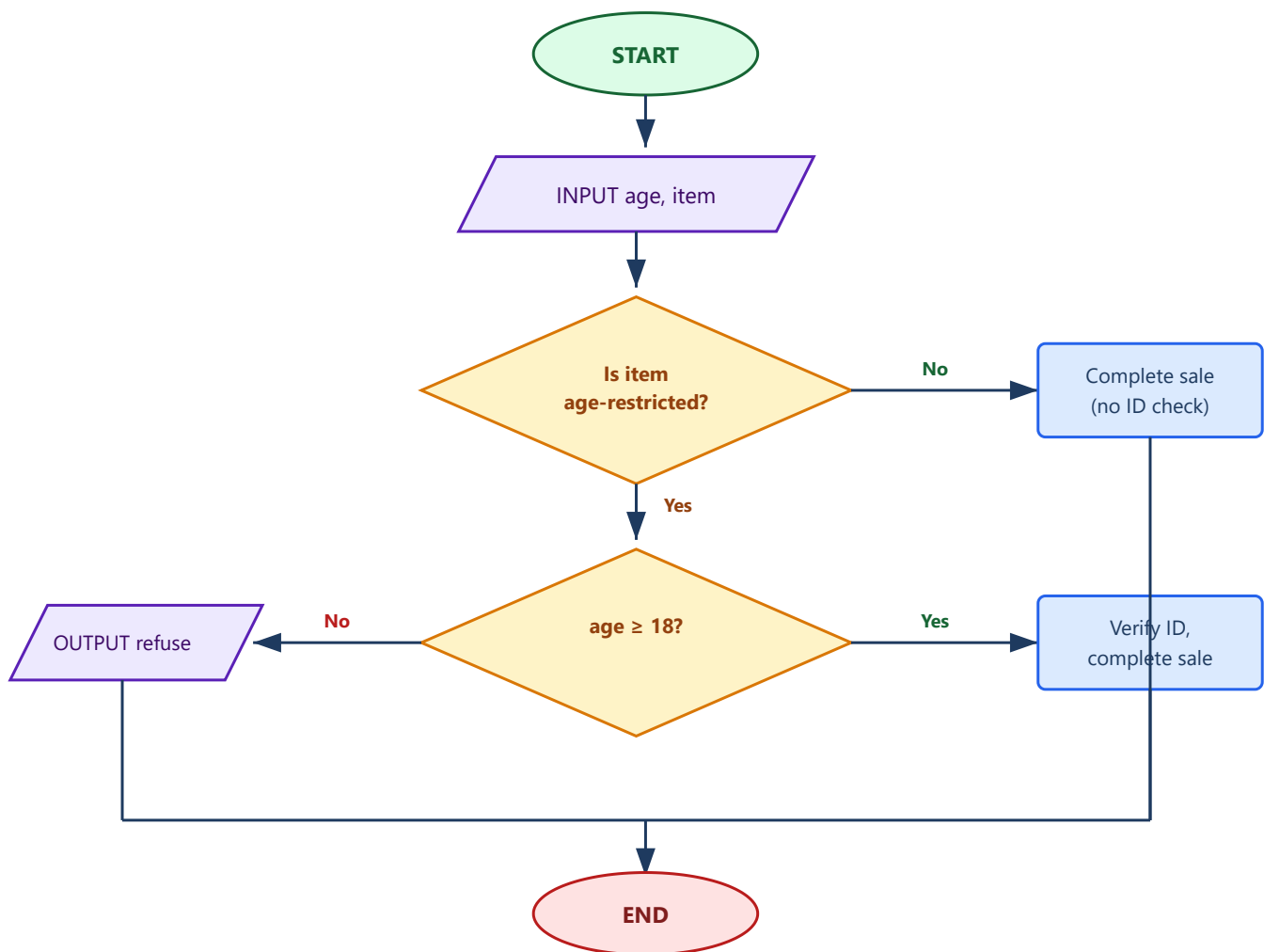
Every arrow in that tree is a Boolean decision. Thinking logically produced the list of questions that must be asked in order; thinking procedurally then chose to answer them in that sequence. Without a careful logical analysis, a game is riddled with edge-case bugs (clicking on own unit while already having one selected; attacking your own unit; moving outside range).

Flowchart Conventions

Flowcharts are the canonical visual notation for logical structure. OCR expects you to know the standard shapes:

Shape	Meaning	Example text
Oval (terminator)	Start or End of the process	"START", "END"
Rectangle (process)	A step / action / calculation	"total ← price × qty"
Diamond (decision)	A Boolean test with two labelled exits	"age ≥ 18?" with Yes/No arrows
Parallelogram (input / output)	Reading input or producing output	"INPUT name", "OUTPUT total"
Arrow	Flow of control between shapes	(labelled with Yes / No on decision exits)

Flowchart Example — Age-Restricted Purchase



The diamonds are the result of logical thinking; the rectangles are the result of procedural thinking; the parallelograms are the result of thinking ahead about inputs and outputs. Even this small diagram uses three of the five elements at once.

Truth Tables and Combined Conditions

Decision points often combine multiple Boolean expressions with **AND**, **OR** and **NOT**. Thinking logically includes reasoning about their interaction. Consider a login routine that permits access only when **credentials are correct AND the account is not locked AND (the user is on the office network OR they have two-factor enabled)**. A truth table lists each combination and the resulting action, and is often the fastest way to see that a chained set of **if** statements has a missing case or a redundant one.

Procedural vs Logical — the Distinction

Thinking procedurally	Thinking logically
What steps and in what order?	Under what conditions does each step execute?
Sequence, sub-procedures, reuse	Boolean tests, branches, truth tables
Produces function bodies and call graphs	Produces <code>if</code> -statements, decision trees, guard clauses
"First validate, then save, then notify."	"Save only if validation passed AND user has write permission."

Exam Tip: Do not conflate logical thinking with "thinking in a sensible order" — that is procedural. Logical thinking specifically concerns **Boolean conditions and the branches they create**. When answering OCR questions, point at a specific decision ("the program must decide whether ...") to demonstrate logical thinking.

2.1.5 Thinking Concurrently

Thinking concurrently is identifying which parts of a problem can be carried out **at the same time (in parallel)** rather than strictly one after another (sequentially), and designing the solution so hardware can exploit that parallelism.

Why Concurrency Matters

For decades, CPUs got faster by raising the clock frequency of a single core. Since the mid-2000s this has plateaued due to heat and power limits, and performance gains have come instead from **more cores**. Modern hardware is concurrent by default:

- **Multi-core CPUs:** desktops have 4–32 cores, servers 64+.
- **GPUs:** thousands of simple cores executing the same instruction on different data (SIMD / SIMT).
- **Cloud compute farms:** tens of thousands of machines cooperating on a single job (MapReduce, Spark).

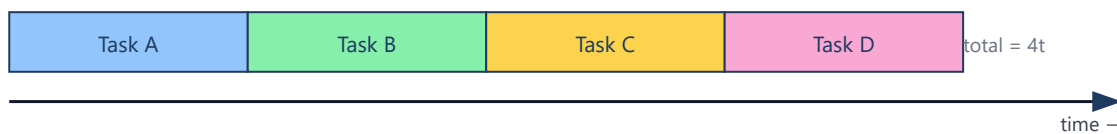
- **Real-time systems:** a car's engine controller must read several sensors simultaneously.
- **Interactive UIs:** the UI thread must stay responsive while background work runs.

Software that does not think concurrently cannot exploit this hardware — it will run at single-core speed on a 16-core machine.

Serial vs Parallel Execution

Serial vs Parallel Execution — Four Tasks

Serial (1 core)



Parallel (4 cores)



The idealised speed-up is linear in the number of cores. In practice, few problems scale that well — some work cannot be parallelised.

Worked Example — Image Filtering is "Embarrassingly Parallel"

Applying a brightness filter to a 4000×3000 photograph means running the same tiny computation on 12 million pixels. The pixels do not depend on each other, so the image can be divided into (say) 16 tiles and each tile given to a different core. When every core finishes its tile, the image is done. This is called an **embarrassingly parallel** workload: almost no coordination is needed between workers.

GPUs go further, dedicating thousands of lanes to such workloads and completing an entire image filter in milliseconds. The same pattern powers deep-learning training (matrix multiplications), ray-traced rendering (rays are independent) and scientific simulation (grid cells).

Amdahl's Law

Amdahl's Law states that the maximum speed-up from parallelising a program is limited by the fraction of the work that *must* remain serial. If a fraction s of the work is serial and $(1-s)$ can be parallelised over n processors, the best-case speed-up is $1 / (s + (1 - s)/n)$.

The key consequence: no matter how many cores you add, if 10% of the program is inherently serial, you cannot get more than a 10× speed-up. Thinking concurrently therefore means not only identifying parallel parts but also **minimising the serial fraction**.

Serial fraction s	Max speed-up ($n \rightarrow \infty$)	Speed-up at $n = 16$
0%	∞	16× (ideal)
5%	20×	~9.1×
10%	10×	~6.4×
50%	2×	~1.9×

Task Parallelism vs Data Parallelism

Two distinct patterns of concurrency appear in OCR-level examples:

- **Data parallelism:** the same operation applied to many items of data simultaneously (e.g. the image filter across pixels, or a spreadsheet recalculating 100,000 cells with the same formula). Ideal for GPUs and SIMD instructions.
- **Task parallelism:** different operations running at the same time, each doing its own work (e.g. the UI thread, a network thread, a logging thread, and an autosave thread in the same editor application). Ideal for multi-core CPUs.

Identifying which pattern fits a workload is part of thinking concurrently. Data parallelism scales almost linearly up to huge core counts when the work is truly independent; task parallelism rarely scales past a handful of cores but keeps interactive programs responsive.

Challenges of Concurrency

Parallelism introduces problems that sequential programs never have:

Challenge	What it is	Mitigation (concept only)
Race condition	Two threads read/write a shared variable in an interleaving that produces an incorrect result.	Protect shared state using a mutex (mutual-exclusion lock) so only one thread accesses it at a time.
Deadlock	Two threads each hold a resource the other needs and neither can proceed.	Acquire locks in a consistent global order; use timeouts.
Shared resource contention	Many threads queue for the same lock / disk / network, throttling throughput.	Partition the data so each thread owns its own shard; use lock-free structures.
Need for synchronisation	One thread must wait until another finishes producing data.	Semaphores , condition variables, barriers, producer/consumer queues.
Non-determinism	Two runs with the same input can schedule threads differently and produce different results or bugs.	Keep shared state minimal; prefer message-passing.

OCR requires only a conceptual awareness of these at 2.1 depth — naming them correctly is enough. Implementation of mutexes and semaphores is beyond this topic.

Worked Example — Rendering an Animation

Imagine an animation studio rendering a 10-second clip at 24 frames per second — 240 frames, each taking five minutes on a single core.

- **Serial approach:** $240 \times 5 \text{ min} = 20 \text{ hours}$ of wall-clock time.
- **Parallel approach:** the frames are independent (each computed from the same scene description). Distribute them across a render farm of 240 cores: in principle ~5 minutes of wall-clock time, plus a small coordination overhead.
- **Interactive use:** the animator's preview window runs on the **main thread** (so it stays responsive to mouse input); rendering runs on background worker threads. The main thread periodically

polls for completed frames and updates the UI.

This is a textbook case of thinking concurrently: spotting a naturally independent workload (one frame does not depend on another), mapping it onto parallel hardware, and keeping the user-facing work on a thread that is never blocked.

Exam Tip: When asked whether a task can be parallelised, ask two questions: (1) *Are the sub-tasks independent?* (no data flows between them) and (2) *Can the hardware actually run them in parallel?* If yes to both, it is a candidate for concurrent design. If sub-tasks depend on each other's output, the problem is inherently sequential.

Connections Between the Five Elements

The five elements are not steps taken in order; they are lenses a developer applies simultaneously. A real design session shifts between them constantly.

The Five Elements Combine in Every Good Design



Mini Case Study — Designing a Ride-Sharing App

A start-up is designing an app like Uber or Lyft. A driver opens the app and waits for requests; a rider opens the app, specifies pickup and destination, is shown a fare estimate, is matched to a nearby driver, watches the car approach on a map and pays at the end. We walk through how **all five elements of computational thinking** inform the design.

(1) Thinking Abstractly

The design team models only the entities that matter: **Rider, Driver, Vehicle, Trip, Location, Payment**. The city's actual road network is abstracted into a weighted graph of junctions and road segments — just as the Tube map abstracts real tunnels. Traffic is represented by time-varying edge weights. Irrelevant detail (building colour, street names, weather) is dropped. A **data abstraction** is introduced so the rest of the system can ask "compute fare for trip T" without knowing whether the pricing engine uses a lookup table, a regression model or a surge multiplier.

(2) Thinking Ahead

- **Inputs/outputs** are specified up-front: rider sends GPS coordinate + destination; server returns list of candidate drivers + ETA + fare; driver accepts/declines.
- **Preconditions**: payment method on file; phone has GPS permission; driver is on-duty and not already on a trip.
- **Caching**: city map tiles are cached on-device (they change rarely); common routes between popular destinations (airport ↔ downtown) are precomputed so fare estimates appear instantly.
- **Edge cases**: no driver available within 20 minutes; GPS jitter; rider cancels mid-trip; driver loses signal inside a tunnel; two riders hit "Book" for the last nearby car at the same instant.
- **Scalability**: the matching service is designed *stateless* from day one so it can be horizontally scaled as the city grows.

(3) Thinking Procedurally

The overall flow is decomposed top-down:

Ride-sharing app
Rider app

```

    Location + destination entry
    Fare estimate display
    Trip tracking UI
    In-app payment
Driver app
    Availability toggle
    Incoming request UI
    Navigation
Backend
    Matching service      (rider requests → driver assignments)
    Pricing service       (compute fare given distance, time, demand)
    Tracking service      (real-time GPS from driver → rider)
    Payment service       (card tokenisation, authorisation, settlement)
    Account / auth service

```

Each leaf is a candidate for an independent team or microservice. **Bottom-up reuse** appears at the leaves: the payment service uses an existing payment gateway library rather than re-implementing card handling.

(4) Thinking Logically

Hundreds of decision points are identified. Examples:

- Is the rider's payment method valid? If no → block booking.
- Is surge pricing active in this area right now? If yes → apply multiplier.
- Are there drivers within 2 km? If no, try 5 km. If no, show "no drivers available".
- Did the driver accept within 15 s? If no → offer the request to the next driver.
- Has the trip actually ended at the destination? Combine GPS proximity AND driver tap on "end trip" to avoid accidental endings.

Each of these becomes a Boolean test in the matching / pricing / trip state machines.

(5) Thinking Concurrently

- **Per-city sharding**: the matching service for London runs independently of the one for Paris, so load in one city does not slow the other.
- **Parallel driver search**: when a rider requests, the server queries many candidate drivers in parallel rather than one at a time.

- **Concurrent UI thread:** the rider's map updates every second with the driver's GPS position *while* the user can still interact with buttons, because network I/O runs on a background thread.
- **Pricing in parallel:** distance, time and surge components are computed by independent workers and combined; none depends on the others' output.
- **Serial bottleneck:** payment authorisation is inherently serial per trip (you cannot charge before the trip ends), which Amdahl's Law warns limits the overall speed-up of a trip-completion workflow.

How the five lenses interact

Design decision	Thinking modes applied
Model city as a weighted graph	Abstractly (remove irrelevant detail)
Cache common airport routes	Ahead (reusability); Abstractly (functional abstraction behind "getFare")
Split backend into services	Procedurally (decomposition); Concurrently (services run on separate hosts)
"Offer to next driver after 15 s"	Logically (decision point); Ahead (edge case: driver does not respond)
Update map on UI thread while network runs in background	Concurrently; Procedurally (two cooperating sequences)
Pricing components computed by parallel workers	Concurrently; Procedurally (sub-procedures per component)
Stateless matching service for scale	Ahead (scalability); Abstractly (hides request state in shared store)

No single row uses only one element. Real software engineering is **the disciplined simultaneous use of all five** — which is why OCR groups them together under 2.1.

Why the Five-Elements Framework is Valuable

Novice programmers often see only one element at a time — they leap to "thinking procedurally" and write code that works for their own test data but fails on edge cases (no thinking ahead), duplicates logic (no abstraction), tangles its `if` branches (weak logical thinking) and blocks the UI while waiting for the network (no concurrent thinking). Practising all five lenses in combination is what distinguishes a competent software engineer from a hobbyist, and it is exactly what the 2.1 specification is trying to develop.

Summary Table of the Five Elements

Element	Core question it answers	Key techniques	Common exam trap
Thinking abstractly	What detail can I ignore?	Representational, procedural, data, functional abstraction; levels of abstraction	Confusing with decomposition
Thinking ahead	What inputs, outputs and future conditions must I prepare for?	I/O spec, preconditions, caching / memoisation, edge cases, scalability	Forgetting it is about anticipation, not just planning the code
Thinking procedurally	What steps, in what order, and what reusable sub-procedures?	Decomposition, top-down stepwise refinement, bottom-up composition	Listing steps without noting reusable sub-procedures
Thinking logically	Under what conditions do branches of the program run?	Boolean conditions, decision trees, flowcharts	Confusing with procedural (order vs condition)
Thinking concurrently	What can run at the same time?	Parallel sub-tasks, synchronisation (concept), Amdahl's Law, race/deadlock awareness	Assuming anything can be parallelised; ignoring the serial fraction

Exam Confidence Points

- Always **define the term** before applying it (one-mark hook in most 2.1 questions).
- For **abstraction** questions, state both what is removed and what is preserved, and say why that suits the user.
- Distinguish **abstraction (remove detail)** from **decomposition (break into parts)** explicitly if both are on the paper.
- For **thinking ahead**, use the OCR sub-list (inputs/outputs, preconditions, caching, edge cases) as a hidden mark scheme.
- Use the shape conventions when drawing flowcharts — diamonds for decisions, parallelograms for I/O. Label every decision exit with Yes/No.
- For **concurrency**, mention **Amdahl's Law** and at least one named hazard (race, deadlock, shared resource). Name a realistic hardware context (multi-core CPU, GPU, render farm).
- In extended-response questions, show how **multiple thinking modes** apply to the same example — examiners reward breadth as well as depth.
- Drawing a **small diagram** (decomposition tree, flowchart, timeline) takes 30 seconds and often secures a mark a paragraph would not.
- Use precise vocabulary: *representational abstraction*, *stepwise refinement*, *precondition*, *memoisation*, *race condition*, *embarrassingly parallel*. These are the terms mark schemes accept.
- Never write "computational thinking means thinking like a computer" — it means formulating problems **so a computer can solve them**, which is the opposite emphasis.